Practice Session 2:

Working with Azure Virtual Machines

© Airowire Networks All Rights Reserved

The information contained in this document is proprietary to Airowire Networks unless otherwise stated and is made available in confidence for the use of clients only. All rights reserved. The information contained in this document must not be used by or disclosed to any other person without the express permission of Airowire Networks. This document may not be copied in whole or in part without the express written consent of Airowire Networks which may be given by contract. Release of the information to any third party is prohibited without prior written consent of Airowire Networks

Introduction to Infrastructure as a Service – Azure Clouds

In this practice session we are going to learn concepts of virtual machines. An Azure virtual machine gives you the flexibility of virtualization without having to buy and maintain the physical hardware that runs it.

Practical session communication!

We run the labs physically in the classroom at RVCE. We use Slack for the running communication between students and lab supervisors.

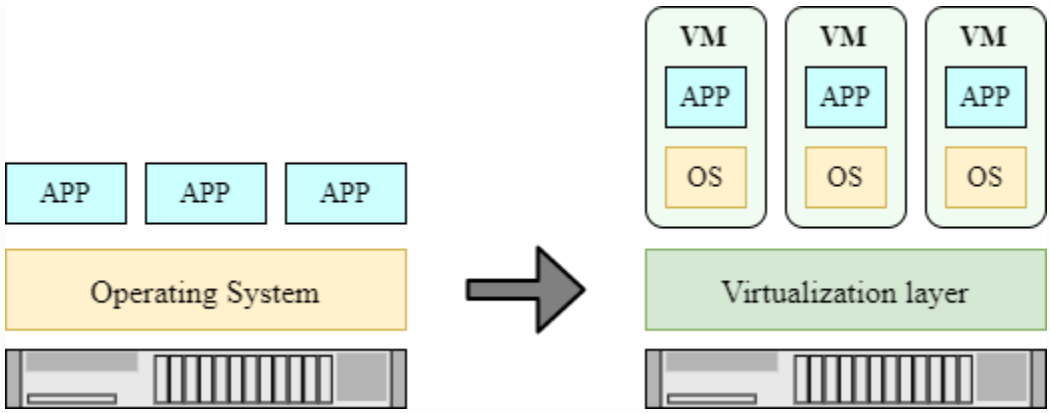
- You must schedule attending the practice sessions at the correct time (based on your chosen lab group). If you complete the lab tasks outside the scheduled practice sessions, we cannot guarantee that you will receive timely support from the lab supervisors.
- We have set up a slack topic for lab discussions.
    - Join the slack channel by clicking here
- When asking questions or support from the lab assistant, please make sure also to provide all needed information, including screenshots, configurations, and even code if needed.

In case of issues check:

- Post you queries on the ***queries*** channel itself.

Introduction

Infrastructure as a Service (IaaS) is a model of Cloud computing, in which virtualized computing resources are provided to users over the internet. In comparison to using physical servers, computing resources can be provisioned **on-demand** and in **real-time** and applications running on the same hardware can be separated into different secure environments, each containing their own OS, software libraries and kernels.



Working with the IaaS model of Cloud usually consists of the following steps:

1. Register an account to access the cloud services
2. Select appropriate virtual machine image to run (Ubuntu, Debian, Windows, etc.)
3. Start a new instance of the selected virtual machine image. Log into the instance as a root user over the internet and configure it to meet your requirements. I.e. install needed software, upload your own application, perform any required configuration actions as you would do in any real computer.
4. As you will lose all your work when the instance is terminated/deleted -- you have three options on how to persist the changes you made:
    1. Save all your configuration steps to a **script** that will launch and configure the instance automatically for you.
    2. Bundle a new **image** from your running instance and next time launch your custom image.
    3. Save the running instance as a **snapshot**, and next time launch new instances from there.

The first option is more flexible as you can easily change the script than bundle a new image if something changes. The second and third options are simpler to use once you have a stable configuration or when launching a large number of instances.

Exercise 2.1. Accessing the Azure cloud services

In this exercise, you will log into the azure cloud. You will verify that you have access to the Azure cloud resources and familiarize yourself with the available cloud offerings from azure.

- Log into https://portal.azure.com/ using your username and password shared by team.
  -
- Familiarize yourself with the available Azure cloud portal functionality.

Exercise 2.2. - Requesting computing resources from the cloud

In this exercise, you will start a Cloud instance (or virtual machine) while specifying its configuration and computing resources available for it.

Step 1: Log in to Azure

1. Open the Azure Portal and log in with your Microsoft account.

Step 2: Create a New Virtual Machine

1. In the **Azure Portal** dashboard, click on **"Create a resource"**.
2. Under **Compute**, select **"Virtual Machine"**.

Step 3: Configure the Basics

1. **Subscription**: Select your subscription
2. **Resource Group**: Select resource group assigned to you.
   a. Example: RG-2
3. **Virtual Machine Name**: Enter a name for your VM.
   a. Example: myLinuxVM
4. **Region**: Choose a region.
   a. Example: East US.
5. **Image**: Select a Linux distribution.
   a. Example: **Ubuntu Server 22.04 LTS**.
6. **Size**: Choose the size of the VM bases on your needs.
   a. Example: **B1S (1 vCPU, 1GB RAM)**.
7. **Authentication Type**: Choose **Password**.
8. **Username**: Enter your desired admin username.
   a. Example: azureuser.
9. **Password**: Set a strong password that meets Azure's complexity requirements:
   a. At least 12 characters.
   b. Includes uppercase, lowercase, numbers, and special characters.

Step 4: Configure Networking

1. **Public Inbound Ports**: Ensure **Allow selected ports** is selected.
2. **Select inbound ports**: Choose **SSH (22),HTTPS(443).**
   a. This allows you to access the VM securely.

Step 5: Review and Create

1. Click **Next** through tabs like Disks, Networking, and Management (keep defaults for simplicity).
2. Click **Review + Create**.
3. Validate your configurations and click **Create** to deploy the VM.

Step 6: Connect to Your VM

1. Once deployment is complete, go to the **Virtual Machines** section in the Azure Portal.
2. Select your VM and copy its **Public IP Address**.(It should be Overview Tab)
3. Open a terminal (or an SSH client like PuTTY for Windows users or current windows supports SSH) and run:

   ```
   ssh username@<Public-IP>
   ```

   Replace username with your admin username (azureuser) and <Public-IP> with your VM's public IP address.

4. When prompted, enter the password you set earlier.

Step 7: Use Your VM

- Once logged in, you can run commands on your Linux VM.
- Example to update packages:

  ```
  sudo apt update && sudo apt upgrade -y
  ```

Step 8: Manage Costs

1. To avoid charges beyond the free tier:
   a. Stop the VM when not in use by selecting **Stop** in the Azure Portal (you can turn it on when needed).
   b. Delete the resource group when you no longer need the VM.

Exercise 1.3. Configuring Cloud Instance & installing software

We will now set up a simplistic Python web application on the instance, setting up the necessary software and dependencies. The application is built with the Flask micro web framework. The app represents a messaging board that stores posted messages into a JSON-structured text file.

1. Install Python Flask webserver on the instance
   - First, fetch the source code of the application with git, storing it in the folder "cloud-computing-2022-lab1":
     - git clone https://github.com/airowireNetworks/devops-web-application.git
     - Enter the application src directory: cd devops-web-application

- This app uses the *pip* package Flask as a dependency. We will create a Python *venv* (virtual environment) for this application and install the packages for the app within the *venv*.
  - Update the package lists about available software and install the *python3-venv* package:
    - `sudo apt update && sudo apt install python3-venv`
  - Create a *venv* named "env", and activate it:
    - `python3 -m venv env`
    - `source env/bin/activate`
    - After activating, you should see the "(env)" appear infront of your username in the terminal session.
  - Use pip to install the necessary dependencies of this application (defined in requirements.txt):
    - `pip install -r requirements.txt`
- Start the Flask web application
  - `flask run --host=0.0.0.0`
    - This will start the Flask server on port 5000.
    - Note: you need to be in the *cloud-computing-2022-lab1* folder!
    - we set the "host=0.0.0.0" parameter to ensures the webserver listens to external traffic too, not only local traffic.
  - (Or, to set it to run in the background, if you wish :)
    - `nohup flask run --host=0.0.0.0 &`
    - *nohup* ensures the program stays running after we log out
    - Note: if you want to kill python program running in the background, you can use `fuser -n tcp -k 5000` - this kills the process listening on port 5000

2. Check that the installation of the Flask web server is successful
   - Try accessing the address of your virtual machine on port 5000 from a browser
     - E.g. http://vm-public-ip:5000
     - This, in theory, should display the `cloud-computing-2022-lab1/templates/home.html` page that's being provided by your web server. (Dont worry if you are unable to get the page. The Ex. 1.4 will give you the ans.)
     - **However**, it does not work at the moment, as by default communication with the cloud instances from outside other than ssh (port 22) is restricted by default.
   - Let's use an alternative approach to verify our web server is running
     - Log into the instance through ssh
     - Use the `wget` command to download the webpage from your server.
     - On the instance command line run `wget localhost:5000`.

        - This should download index.html file into the currently active directory, which among other HTML codes should contain a string "Message board".
    - Use `less index.html` command to check the downloaded file content from the command line
    - Alternatively you can use command line web-browser `sudo apt install lynx` and `lynx localhost:5000`
3. Modify the current home.html file at `devops-web-application/templates/home.html` to change the web page content.
    - How exactly you change its content is up to you, but it should at least contain your **Full Name**, so it is possible to visually see that you have modified it.
    - To deploy the changes, you need stop and re-start the Flask server! See Ex 1.4 for the relevant commands.
    - Command line file editor `nano` can be used to modify file contents.

Exercise 1.4. Creating Security group to enable access through port 5000 and 80

Step 1: Log in to Azure Portal

- Go to https://portal.azure.com
- Sign in using your Azure credentials

Step 2: Locate Your Virtual Machine

- In the left-hand menu, click **"Virtual Machines"**
- Select the VM you want to modify

Step 3: Open the Networking Tab

- From the VM's management page, click **"Networking"** in the left-side menu

---

Step 4: Add an Inbound Port Rule for Port 5000

- In the **Inbound port rules** section, click **"Add inbound port rule"**
- Set the following values:

- **Source**: Any
- **Source port ranges**: *
- **Destination**: Any
- **Destination port ranges**: 5000
- **Protocol**: TCP
- **Action**: Allow
- **Priority**: Leave default
- **Name**: Allow-Port-5000

- Click **Add**

Step 5: Add an Inbound Port Rule for Port 80

- Repeat the same steps as above
- Change only the **Destination port ranges** to 80
- Use a different name, such as Allow-Port-80
- Click **Add**

Step 6: Verify Rules Are Applied

- Ensure both new rules (Allow-Port-5000 and Allow-Port-80) appear in the inbound rules list
- Confirm both rules are set to "Allow" and apply to TCP traffic from any source (0.0.0.0/0)

Step 7: Access the Web Server

- Go to the **Overview** tab of your VM
- Copy the **public IP address**
- In your web browser, enter:
  - http://<your-public-ip>:5000
- If your web server is running and listening on those ports, it should load correctly